

SQL + MongoDB — Side by Side

Backend Engineer ke liye Database Course • PDF 2 of 5 • Parts 5–8

Part	Topic	Kyun important
5	Sort, Limit, Pagination (Offset vs Cursor)	Har list API mein; interview favourite
6	JOIN vs \$lookup, \$unwind, \$arrayElemAt	Relational data padhna — sabse zyada poocha jaata hai
7	GROUP BY vs \$group (COUNT/SUM/AVG/MIN/MAX)	Reports, dashboards, analytics
8	Aggregation Pipeline (poora example)	MongoDB ka asli power; SQL mein CTE se compare

► DATA WAHI HAI JO PDF 1 KE SECTION 2.0 MEIN LOAD KIYA

Bas **ek naya user** add karo — Anita, jiska **koi order nahi** hai. LEFT JOIN aur \$lookup ka difference isi se dikhega.

SQL (dono)

```
INSERT INTO users (user_id, name, city, email) VALUES ('1106', 'Anita', 'Nagpur', 'anita@shop.in');
```

MongoDB

```
db.users.insertOne({ userId: "1106", name: "Anita", city: "Nagpur", email: "anita@shop.in" })
```

Quick yaad: **users** 1103 Sonu, 1104 Priya, 1105 Rahul (email NULL), 1106 Anita (koi order nahi). **orders** O1001 (1103, 1500, PAID), O1002 (1104, 900, PENDING), O1003 (1103, 280, CANCELLED), O1004 (1105, 1000, PAID). **products** P101 Steel Pipe 500, P102 PVC Pipe 200, P103 Elbow Joint 80, P104 Copper Pipe 900. Mongo mein order items **embedded** hain (`orders.items[]`), SQL mein alag table `order_items`.

PART 5 — Sort, Limit, Pagination

5.1 ORDER BY → .sort()

► CONCEPT

Results ko kisi field ke hisaab se arrange karna. 'Sabse mehenge products pehle', 'latest orders pehle'.

► WHY?

Bina ORDER BY ke database **koi order guarantee nahi karta**. Aaj insert order mein aa raha hai, kal index/update ke baad alag order mein aa sakta hai. UI mein list hamesha kisi defined order mein chahiye.

SQL (dono)

```
SELECT product_id, name, price FROM products ORDER BY price DESC;    -- mehenga pehle
SELECT product_id, name, price FROM products ORDER BY price ASC;      -- sasta pehle (ASC default)

-- Multiple fields: pehle category, phir usmein price high→low
SELECT product_id, category_id, price FROM products
ORDER BY category_id ASC, price DESC;
```

MongoDB

```
db.products.find({}, { productId: 1, name: 1, price: 1, _id: 0 }).sort({ price: -1 })
db.products.find({}, { productId: 1, name: 1, price: 1, _id: 0 }).sort({ price: 1 })

db.products.find({}, { productId: 1, categoryId: 1, price: 1, _id: 0 })
.sort({ categoryId: 1, price: -1 })
```

Output

ORDER BY price DESC	ORDER BY category_id, price DESC
P104 Copper Pipe 900	P104 C10 900
P101 Steel Pipe 500	P101 C10 500
P102 PVC Pipe 200	P102 C10 200
P103 Elbow Joint 80	P103 C20 80

► LINE-BY-LINE

- **DESC** = **-1** (bada → chhota). **ASC** = **1** (chhota → bada).
- Multiple fields: pehle field se sort, tie hone par doosra. Mongo mein **object ki key order** hi sort priority hai.
- Strings alphabetically sort hoti hain — **"10" < "9"** (string compare). Numbers ko number type mein hi store karo.

► NULL / MISSING VALUES KAHAN JAATE HAIN? (PG VS MYSQL VS MONGO ALAG!)

Database	ASC mein NULL	DESC mein NULL	Control
PostgreSQL	Last	First	NULLS FIRST / NULLS LAST
MySQL	First	Last	Trick: ORDER BY email IS NULL, email
MongoDB	First (null/missing sabse chhote)	Last	Pehle \$match se hatao, ya aggregation

PostgreSQL

```
SELECT name, email FROM users ORDER BY email ASC NULLS LAST;    -- PostgreSQL
```

COMMON MISTAKES

- Sort bhoolna aur maan lena ki 'insert order' mein aayega. SQL aur Mongo dono mein koi guarantee nahi.
- Non-unique field pe sort karke paginate karna (5.4 mein detail) — same record do pages pe, ya miss.
- Mongo mein bina index ke bade result ko sort karna — in-memory sort ki limit (100 MB). MongoDB 6.0+ zaroorat pe disk use kar leta hai (`allowDiskUse`), lekin slow. Index lagao (Part 10).

INTERVIEW QUESTION

Q: SQL query mein ORDER BY nahi diya — result kis order mein aayega?

Answer: Undefined. DB jo bhi order sabse efficient ho (storage order, index order, parallel scan) mein dega, aur ye badal sakta hai. Order chahiye to ORDER BY / `.sort()` compulsory.

MUST KNOW	GOOD TO KNOW	NOT NECESSARY RIGHT NOW
• ORDER BY / sort, ASC=1, DESC=-1 • Multi-field sort • Bina sort = koi guarantee nahi	• NULL ordering difference • Mongo 100 MB in-memory sort limit	• Sort algorithm internals

5.2 LIMIT → `.limit()` • OFFSET → `.skip()`

SQL (dono)

```
-- Top 2 mehenge products
SELECT name, price FROM products ORDER BY price DESC LIMIT 2;

-- Page 2 (2 per page) → pehle 2 chhodo
SELECT name, price FROM products ORDER BY price DESC LIMIT 2 OFFSET 2;
-- MySQL short form bhi hai: LIMIT 2, 2    (offset, count – ULTA order, confusing!)
```

MongoDB

```
db.products.find({}, { name: 1, price: 1, _id: 0 }).sort({ price: -1 }).limit(2)
db.products.find({}, { name: 1, price: 1, _id: 0 }).sort({ price: -1 }).skip(2).limit(2)
```

Output

```
Page 1:  Copper Pipe 900, Steel Pipe 500
Page 2:  PVC Pipe 200,    Elbow Joint 80
```

► LINE-BY-LINE

- **LIMIT** `n` = max `n` rows. **OFFSET** `k` = pehle `k` rows chhod do.
- Formula: page `p` (1 se shuru), size `s` → **OFFSET** $(p-1)*s$ **LIMIT** `s` → `.skip((p-1)*s).limit(s)`.
- Mongo cursor pe `.limit().skip().sort()` kisi bhi order mein likho — DB hamesha **sort** → **skip** → **limit** hi apply karta hai. SQL mein clause order fixed hai: **WHERE** → **ORDER BY** → **LIMIT** → **OFFSET**. (Aggregation pipeline mein order matter karta hai — Part 8.)

MUST KNOW	GOOD TO KNOW	NOT NECESSARY RIGHT NOW
- LIMIT/OFFSET ↔ limit/skip - Page formula	MySQL LIMIT a, b syntax	—

5.3 Offset Pagination — simple, lekin problems ke saath

► REAL EXAMPLE

Admin panel: `GET /orders?page=3&limit=20`, latest orders pehle.

SQL (dono)

```
SELECT order_id, user_id, total, status, created_at
FROM orders
ORDER BY created_at DESC, order_id DESC
LIMIT 20 OFFSET 40;           -- page 3

SELECT COUNT(*) FROM orders; -- total pages dikhane ke liye
```

MongoDB

```
db.orders.find({}).sort({ createdAt: -1, _id: -1 }).skip(40).limit(20)
db.orders.countDocuments({})
```

► PROBLEM 1 — DEEP PAGES SLOW HOTE HAIN

OFFSET 100000 ka matlab: DB ko **1,00,020 rows padhni padengi**, 1,00,000 phenk deni padengi, 20 return karni hain. Index ho tab bhi skip kiye hue entries walk karne padte hain. Page 5000 pe query seconds le sakti hai.

Output

```
Page 1      → 20 rows padhi
Page 100    → 2,000 rows padhi
Page 5,000  → 1,00,000 rows padhi ← time linearly badhta hai
```

► PROBLEM 2 — DATA BADALNE PE DUPLICATE / MISSING RECORDS

Output

```
User page 1 dekh raha hai: [020 ... 001] (20 orders, latest pehle)
Tabhi ek naya order 021 aa gaya.
User page 2 maangta hai → OFFSET 20:
  Ab list: [021, 020 ... 002, 001, ...] → 20 chhoda to 001 page 2 pe DOBARA aayega.
(Delete hua to ulta — ek record kabhi dikhega hi nahi.)
```

► PROBLEM 3 — COUNT(*) BHI MEHENGA

Bade table pe har request ke saath `COUNT(*)` / `countDocuments` full index/table scan kar sakta hai. Isliye bade apps 'Page 1 of 58,392' ki jagah 'Next' button dikhate hain, ya approximate count.

MUST KNOW	GOOD TO KNOW	NOT NECESSARY RIGHT NOW
- Offset slow deep pages pe - Live data mein duplicates/miss - COUNT ka cost	estimatedDocumentCount (Mongo)	—

5.4 Cursor (Keyset) Pagination — production ka standard

► CONCEPT

Page number ki jagah bolo: **'last dekha hua record ye tha, uske baad ke 20 do'**. DB index mein seedha us point pe jump karta hai — skip karne ki zaroorat hi nahi.

Output

```
Offset: "Pehle 40 ginke chhodo, phir 20 do"      → ginna padta hai
Cursor: "created_at < '2026-09-05 09:15' wale 20 do" → index se seedha jump
```

► TIE-BREAKER KYUN ZAROORI HAI

Do orders ka **created_at** exactly same ho sakta hai. Sirf **created_at < X** kiya to same time wale orders chhoot jaayenge. Isliye sort mein ek **unique field** bhi add karo — (**created_at, order_id**) / (**createdAt, _id**). Ab har record ki position unique hai.

► SQL

SQL (dono)

```
-- Page 1 (koi cursor nahi)
SELECT order_id, total, status, created_at
FROM orders
ORDER BY created_at DESC, order_id DESC
LIMIT 2;
-- 01004 | 2026-09-07 18:45
-- 01003 | 2026-09-05 09:15    ← last row = cursor

-- Page 2: cursor = (2026-09-05 09:15, '01003')
SELECT order_id, total, status, created_at
FROM orders
WHERE created_at < '2026-09-05 09:15:00'
   OR (created_at = '2026-09-05 09:15:00' AND order_id < '01003')
ORDER BY created_at DESC, order_id DESC
LIMIT 2;
-- 01002 | 2026-09-03 11:30
-- 01001 | 2026-09-01 10:00
```

PostgreSQL

```
-- Short form: row value comparison (same meaning, index bhi use hota hai)
WHERE (created_at, order_id) < ('2026-09-05 09:15:00', '01003')
```

MySQL bhi row comparison syntax support karta hai, lekin purane versions mein ye index efficiently use nahi karta tha — MySQL mein expanded **OR** wala form safe choice hai.

► MONGODB

MongoDB

```
// Page 1
db.orders.find({}).sort({ createdAt: -1, _id: -1 }).limit(2)

// Page 2 – cursor = { createdAt: ISODate("2026-09-05T09:15:00Z"), _id: ObjectId("...01003 ka...") }
db.orders.find({
  $or: [
    { createdAt: { $lt: ISODate("2026-09-05T09:15:00Z") } },
    { createdAt: ISODate("2026-09-05T09:15:00Z"), _id: { $lt: ObjectId("...") } }
  ]
}).sort({ createdAt: -1, _id: -1 }).limit(2)

// Shortcut: agar sirf _id se sort kar sakte ho (ObjectId roughly time-ordered hai)
db.orders.find({ _id: { $lt: ObjectId("...last _id...") } }).sort({ _id: -1 }).limit(20)
```

► INDEX — ISKE BINA CURSOR PAGINATION KA FAYDA NAHI

SQL (dono)

```
CREATE INDEX idx_orders_created ON orders (created_at DESC, order_id DESC);
```

MongoDB

```
db.orders.createIndex({ createdAt: -1, _id: -1 })
```

► NODE.JS — COMPLETE API (OPAQUE CURSOR KE SAATH)

Node.js

```
// Cursor ko base64 string bana ke client ko do – client ko andar ka structure na pata ho
const encode = (o) => Buffer.from(JSON.stringify(o)).toString("base64url");
const decode = (s) => JSON.parse(Buffer.from(s, "base64url").toString());

// ----- PostgreSQL (pg) -----
async function listOrdersPg({ limit = 20, cursor }) {
  limit = Math.min(Number(limit) || 20, 100); // max limit enforce karo
  const params = [limit + 1]; // +1 → "aur hai?" pata chalega
  let where = "";
  if (cursor) {
    const { createdAt, orderId } = decode(cursor);
    params.push(createdAt, orderId);
    where = "WHERE (created_at, order_id) < ($2, $3)";
  }
  const { rows } = await pool.query(
    `SELECT order_id, user_id, total, status, created_at FROM orders
    ${where} ORDER BY created_at DESC, order_id DESC LIMIT $1`, params);

  const hasMore = rows.length > limit;
  const items = hasMore ? rows.slice(0, limit) : rows;
  const last = items[items.length - 1];
  return {
    items,
    nextCursor: hasMore ? encode({ createdAt: last.createdAt, orderId: last.order_id }) : null,
  };
}

// ----- MongoDB driver -----
async function listOrdersMongo({ limit = 20, cursor }) {
  limit = Math.min(Number(limit) || 20, 100);
  const filter = {};
  if (cursor) {
    const { createdAt, id } = decode(cursor);
    const t = new Date(createdAt), oid = new ObjectId(id);
    filter.$or = [{ createdAt: { $lt: t } }, { createdAt: t, _id: { $lt: oid } }];
  }
  const docs = await orders.find(filter).sort({ createdAt: -1, _id: -1 })
    .limit(limit + 1).toArray();

  const hasMore = docs.length > limit;
  const items = hasMore ? docs.slice(0, limit) : docs;
  const last = items.at(-1);
  return { items, nextCursor: hasMore ? encode({ createdAt: last.createdAt, id: last._id }) : null };
}

// ----- Prisma (built-in cursor support) -----
await prisma.order.findMany({
  take: 20,
  ...(cursor && { cursor: { orderId: cursor }, skip: 1 }), // skip:1 → cursor wala row khud nahi
  orderBy: [{ createdAt: "desc" }, { orderId: "desc" }],
});
```

Output

```
GET /orders?limit=2
{ "items": [ {01004...}, {01003...} ], "nextCursor": "eyJjcmVhdGVkQXQiOi..." }

GET /orders?limit=2&cursor=eyJjcmVhdGVkQXQiOi...
{ "items": [ {01002...}, {01001...} ], "nextCursor": null } ← last page
```

► COMPARISON

	Offset	Cursor (Keyset)
Speed deep pages pe	Slow (O(offset))	Hamesha fast (index seek)
Live data (insert/delete)	Duplicates / missing	Stable

	Offset	Cursor (Keyset)
'Page 57 pe jao'	Haan	Nahi — sirf next/prev
Total pages dikhana	Easy (COUNT)	Usually nahi
Implement karna	Bahut easy	Thoda kaam (tie-breaker, encode)
Best for	Admin tables, chhota data	Feeds, infinite scroll, mobile apps, public APIs, bade tables

COMMON MISTAKES

- Cursor pagination mein unique tie-breaker bhoolna → same timestamp wale records miss.
- Sort order aur cursor condition mismatch (sort DESC, condition >).
- Client ko `limit=100000` bhejne dena — max limit hamesha server pe cap karo.
- Cursor pagination lagaya lekin index nahi banaya — fir bhi full scan + sort.
- Cursor mein raw internal data bhejna jisse client khelta hai. Opaque (base64) rakho; sensitive ho to sign karo.

INTERVIEW QUESTION

Q: Instagram-type feed mein pagination kaise implement karoge? Offset kyun nahi?

Answer: Cursor/keyset pagination: (`created_at`, `id`) pe composite index, har response ke saath last item ka cursor, next request mein `WHERE (created_at, id) < cursor ORDER BY ... LIMIT n+1`. Offset deep pages pe linearly slow hota hai aur naye posts aane se duplicates dikhte hain. Cursor $O(\log n)$ seek + stable results deta hai. Tradeoff: random page jump nahi kar sakte.

MUST KNOW	GOOD TO KNOW	NOT NECESSARY RIGHT NOW
• Offset vs cursor — tradeoffs bol sako • Tie-breaker (unique field) • <code>limit+1</code> trick for <code>hasMore</code> • Composite index on sort fields • Max limit cap	• Opaque base64 cursor • Prisma cursor API • Row value comparison (PG)	• Backward (prev page) cursor ki poori implementation • Seek method ke planner internals

Part 5 — Memory Table

SQL	MongoDB	Note
ORDER BY f DESC	<code>.sort({ f: -1 })</code>	Bina sort = random order
ORDER BY a, b DESC	<code>.sort({ a: 1, b: -1 })</code>	Key order = priority
LIMIT n	<code>.limit(n)</code>	
OFFSET k	<code>.skip(k)</code>	Deep pages slow
COUNT(*)	<code>countDocuments({})</code>	Bade data pe mehenga
WHERE (t, id) < (x, y)	<code>\$or: [{t:{\$lt:x}}, {t:x, _id:{\$lt:y}}]</code>	Cursor pagination
NULLS LAST (PG)	—	NULL position DB pe depend

PART 6 — JOIN vs \$lookup

Ye course ka **sabse important** part hai. SQL mein data normalize karke alag tables mein rakhte hain, isliye padhte waqt unhe **jodna** padta hai. MongoDB mein embedding se aksar JOIN ki zaroorat hi nahi padti — lekin jab references use kiye, tab **\$lookup** chahiye.

6.1 JOIN kya hai? (INNER JOIN)

► CONCEPT

Do tables ki rows ko ek common column (**user_id**) ke basis pe **jodna**. Orders table mein sirf **user_id = 1103** hai — customer ka **naam** users table mein hai. Order list pe naam dikhana hai → JOIN.

► REAL-WORLD EXAMPLE

Do registers: orders register mein 'customer no. 1103', customers register mein '1103 = Sonu, Mumbai'. Clerk har order ke liye customers register kholke naam dekhta hai — yahi JOIN hai.

► SQL

SQL (dono)

```
SELECT
  u.name,
  o.order_id,
  o.total
FROM users u
JOIN orders o
  ON u.user_id = o.user_id
ORDER BY o.order_id;
```

Output

name	order_id	total
Sonu	01001	1500
Priya	01002	900
Sonu	01003	280
Rahul	01004	1000

(4 rows) ← Anita (1106) nahi aayi — uska koi order nahi

► LINE-BY-LINE

- **FROM users u** — users table, short naam (alias) **u**.
- **JOIN orders o** — orders table jodo, alias **o**. **JOIN** = **INNER JOIN** (same cheez).
- **ON u.user_id = o.user_id** — **join condition**: kaunsi row kis row se judegi.
- **u.name, o.order_id** — dono tables ke columns ek saath select. Alias se pata chalta hai kaunsa column kis table ka.
- Sonu **do baar** aaya — kyunki uske 2 orders hain. JOIN ka result = har matching pair ki ek row.
- **INNER** = sirf wo rows jinka match **dono** taraf mila. Anita ka koi order nahi → gayab.

► MONGODB — \$LOOKUP (ORDERS SE SHURU KARKE USER JODO)

MongoDB

```
db.orders.aggregate([
  {
    $lookup: {
      from: "users",           // kis collection se jodna hai
      localField: "userId",    // orders ka field
      foreignField: "userId",  // users ka field
      as: "user"              // result kis naye field mein rakhna hai (ARRAY)
    }
  },
  { $project: { _id: 0, orderId: 1, total: 1, user: 1 } },
  { $sort: { orderId: 1 } }
])
```

Output

```
[
  { orderId: "01001", total: 1500,
    user: [ { _id: ObjectId("..."), userId: "1103", name: "Sonu", city: "Mumbai",
      email: "...", address: {...} } ] },
  { orderId: "01002", total: 900,
    user: [ { _id: ObjectId("..."), userId: "1104", name: "Priya", ... } ] },
  { orderId: "01003", total: 280,
    user: [ { _id: ObjectId("..."), userId: "1103", name: "Sonu", ... } ] },
  { orderId: "01004", total: 1000,
    user: [ { _id: ObjectId("..."), userId: "1105", name: "Rahul", ... } ] }
]
```

► LINE-BY-LINE

- **aggregate([...])** — \$lookup sirf aggregation pipeline mein chalta hai, **find()** mein nahi.
- **from** — doosri collection (same database mein honi chahiye).
- **localField / foreignField** — SQL ka **ON o.user_id = u.user_id**. Condition equality hi hoti hai (is simple form mein).
- **as: "user"** — matched documents is field mein **hamesha ARRAY** ke roop mein aate hain. Ek match = 1 element ka array. Koi match nahi = **[]**.
- SQL ki tarah flat rows nahi bani — har order ek document hi raha, user uske andar nest ho gaya. **Ye sabse bada conceptual difference hai.**

SQL JOIN vs \$lookup — shape ka farak

- SQL JOIN: 1 user × 2 orders = **2 flat rows** (user data repeat hota hai).
- \$lookup (users se shuru): 1 user document, andar **orders: [{...}, {...}]** — **1 nested document**.
- Flat rows chahiye? → **\$unwind** (6.4). Ek object chahiye? → **\$arrayElemAt** (6.3).

MUST KNOW	GOOD TO KNOW	NOT NECESSARY RIGHT NOW
• INNER JOIN syntax + ON • Aliases • \$lookup ke 4 fields • \$lookup ka result ARRAY hota hai	• JOIN = INNER JOIN	• Join algorithms (nested loop / hash / merge)

6.2 LEFT JOIN — aur \$lookup by default LEFT JOIN jaisa hai

► REQUIREMENT

'Saare users dikhao, unke orders ke saath — jinke orders nahi hain wo bhi' (Anita ko marketing email bhejna hai).

SQL (dono)

```
SELECT u.user_id, u.name, o.order_id, o.total
FROM users u
LEFT JOIN orders o ON u.user_id = o.user_id
ORDER BY u.user_id;
```

Output

user_id	name	order_id	total
1103	Sonu	01001	1500
1103	Sonu	01003	280
1104	Priya	01002	900
1105	Rahul	01004	1000
1106	Anita	NULL	NULL

← LEFT table (users) ki har row aayegi

MongoDB

```
db.users.aggregate([
  { $lookup: { from: "orders", localField: "userId", foreignField: "userId", as: "orders" } },
  { $project: { _id: 0, userId: 1, name: 1, "orders.orderId": 1, "orders.total": 1 } }
])
```

Output

```
[
  {
    userId: "1103",
    name: "Sonu",
    orders: [
      { orderId: "01001", total: 1500 },
      { orderId: "01003", total: 280 }
    ]
  },
  {
    userId: "1104",
    name: "Priya",
    orders: [ { orderId: "01002", total: 900 } ]
  },
  {
    userId: "1105",
    name: "Rahul",
    orders: [ { orderId: "01004", total: 1000 } ]
  },
  {
    userId: "1106",
    name: "Anita",
    orders: []
  }
]
```

← empty array, document gayab NAHI hua

► PATTERN: 'JINKA MATCH NAHI HAI' (ANTI-JOIN)

SQL (dono)

```
-- Jin users ne kabhi order nahi kiya
SELECT u.user_id, u.name
FROM users u
LEFT JOIN orders o ON u.user_id = o.user_id
WHERE o.order_id IS NULL;
-- → 1106 Anita

-- Same, NOT EXISTS se (aksar zyada readable, aur NULL-safe)
SELECT user_id, name FROM users u
WHERE NOT EXISTS (SELECT 1 FROM orders o WHERE o.user_id = u.user_id);
```

MongoDB

```
db.users.aggregate([
  { $lookup: { from: "orders", localField: "userId", foreignField: "userId", as: "orders" } },
  { $match: { orders: { $size: 0 } } }, // ya { orders: [] }
  { $project: { _id: 0, userId: 1, name: 1 } }
])
// → { userId: "1106", name: "Anita" }
```

► INNER JOIN MONGO MEIN KAISE?

MongoDB

```
db.users.aggregate([
  { $lookup: { from: "orders", localField: "userId", foreignField: "userId", as: "orders" } },
  { $match: { orders: { $ne: [] } } } // empty array wale hata do = INNER
])
```

► BAAKI JOIN TYPES (BAS NAAM AUR MATLAB)

JOIN	Kya deta hai	Mongo equivalent	Kitna use hota hai
INNER	Sirf dono taraf match	<code>\$lookup + \$match { arr: { \$ne: [] } }</code>	Bahut
LEFT	Left ki saari + match (warna NULL)	<code>\$lookup (default)</code>	Bahut
RIGHT	Right ki saari	Tables swap karke LEFT likho	Kam
FULL OUTER	Dono ki saari	Direct nahi — <code>\$unionWith</code> se banaya ja sakta	Kam (MySQL support hi nahi karta)
CROSS	Har row × har row	—	Rare
SELF	Table khud se (employee → manager)	<code>\$lookup same collection se / \$graphLookup</code>	Kabhi kabhi

COMMON MISTAKES

- LEFT JOIN ke baad WHERE mein right table ki condition: `LEFT JOIN orders o ... WHERE o.status = 'PAID'` — Anita ki row (status NULL) filter ho gayi, LEFT JOIN chupke se INNER ban gaya! Condition ON mein daalo: `LEFT JOIN orders o ON u.user_id = o.user_id AND o.status = 'PAID'`.
- Mongo mein `$lookup` ke baad `$unwind` kiya (default options) — Anita gayab. LEFT behaviour chahiye to `preserveNullAndEmptyArrays: true` (6.4).
- MySQL mein FULL OUTER JOIN likhna — syntax error. LEFT JOIN UNION RIGHT JOIN se banate hain.

INTERVIEW QUESTION

Q: INNER aur LEFT JOIN mein difference? MongoDB `$lookup` kaunsa hai?

Answer: INNER sirf matching pairs deta hai; LEFT, left table ki har row deta hai aur match na mile to right columns NULL. `$lookup` LEFT OUTER JOIN jaisa hai — input document hamesha rehta hai, match na mile to `as` field empty array. INNER chahiye to baad mein empty arrays `$match` se hatao.

MUST KNOW	GOOD TO KNOW	NOT NECESSARY RIGHT NOW
- INNER vs LEFT <code>\$lookup</code> = LEFT (empty array) - Anti-join pattern - LEFT JOIN + WHERE trap	- NOT EXISTS - RIGHT/FULL/SELF naam aur matlab	<code>\$graphLookup</code> <code>\$unionWith</code> details

6.3 \$arrayElemAt — array se ek object nikalna

► PROBLEM

Order ka sirf **ek** user hota hai, lekin \$lookup ne `user: [ { ... } ]` array diya. Frontend ko `order.user.name` chahiye, `order.user[0].name` nahi.

MongoDB

```
db.orders.aggregate([
  { $lookup: { from: "users", localField: "userId", foreignField: "userId", as: "user" } },
  { $addFields: { user: { $arrayElemAt: ["$user", 0] } } }, // array → pehla element
  { $project: { _id: 0, orderId: 1, total: 1, customer: "$user.name", city: "$user.city" } }
])
```

Output

```
[
  { orderId: "01001", total: 1500, customer: "Sonu", city: "Mumbai" },
  { orderId: "01002", total: 900, customer: "Priya", city: "Pune" },
  { orderId: "01003", total: 280, customer: "Sonu", city: "Mumbai" },
  { orderId: "01004", total: 1000, customer: "Rahul", city: "Delhi" }
]
```

► LINE-BY-LINE

- `$addFields` (alias `$set`) — naya field add karo ya existing overwrite karo. Yahan `user` ko overwrite kiya.
- `{ $arrayElemAt: ["$user", 0] }` — `"$user"` array ka index 0. `-1` = last element.
- Array khaali ho to result field **missing** ho jaata hai (error nahi). LEFT JOIN behaviour bana rehta hai.
- `customer: "$user.name"` — \$project mein rename + nested value nikalna = SQL `u.name AS customer`.
- Shortcut: `{ $first: "$user" }` (MongoDB 4.4+) — same kaam.

Kab use karo: jab relationship **many-to-one / one-to-one** ho (order → user, product → category). Result ka shape nahi badalta — har order ek hi document rehta hai.

MUST KNOW	GOOD TO KNOW	NOT NECESSARY RIGHT NOW
• <code>\$arrayElemAt [arr, 0]</code> • <code>\$addFields / \$set</code>	• <code>\$first</code> • Empty array → field missing	• —

6.4 \$unwind — array ko rows mein todna

► CONCEPT

`$unwind` ek document ko uske array ke **har element ke liye ek document** mein tod deta hai. SQL JOIN jaisa flat result isi se milta hai. Order O1003 mein 2 items hain → unwind ke baad 2 documents.

MongoDB

```
db.orders.aggregate([
  { $match: { orderId: "01003" } },
  { $unwind: "$items" },
  { $project: { _id: 0, orderId: 1, items: 1 } }
])
```

Output

```
PEHLE (1 document):
{ orderId: "01003", items: [ { productId: "P102", quantity: 1, price: 200 },
                             { productId: "P103", quantity: 1, price: 80 } ] }

BAAD (2 documents):
{ orderId: "01003", items: { productId: "P102", quantity: 1, price: 200 } }
{ orderId: "01003", items: { productId: "P103", quantity: 1, price: 80 } }
    ↑ orderId copy hua, items ab array nahi, single object hai
```

► UNWIND + \$LOOKUP = SQL KA MULTI-TABLE JOIN

Requirement: 'Har order line mein product ka naam dikhao' — SQL mein **orders** → **order_items** → **products**.

SQL (dono)

```
SELECT o.order_id, p.name AS product, oi.quantity, oi.price,
       oi.quantity * oi.price AS line_total
FROM orders o
JOIN order_items oi ON oi.order_id = o.order_id
JOIN products p ON p.product_id = oi.product_id
ORDER BY o.order_id, p.name;
```

MongoDB

```
db.orders.aggregate([
  { $unwind: "$items" }, // 1 row per item
  { $lookup: { from: "products", localField: "items.productId",
                foreignField: "productId", as: "product" } },
  { $unwind: "$product" }, // array → object
  { $project: {
    _id: 0, orderId: 1,
    product: "$product.name",
    quantity: "$items.quantity",
    price: "$items.price",
    lineTotal: { $multiply: ["$items.quantity", "$items.price"] }
  } },
  { $sort: { orderId: 1, product: 1 } }
])
```

Output

order_id	product	quantity	price	line_total
01001	Steel Pipe	3	500	1500
01002	Copper Pipe	1	900	900
01003	Elbow Joint	1	80	80
01003	PVC Pipe	1	200	200
01004	Steel Pipe	2	500	1000

(Mongo mein same data, documents ki shape mein)

► LINE-BY-LINE

- 1st **\$unwind: "\$items"** — SQL ke **JOIN order_items** jaisa (har item ek row). Mongo mein items already embedded the, isliye lookup nahi, sirf unwind.
- **localField: "items.productId"** — unwind ke baad **items** object hai, uska productId.
- 2nd **\$unwind: "\$product"** — \$lookup ka 1-element array → object. (Yahan **\$arrayElemAt** bhi chalta.)
- **\$multiply** — computed column, SQL **oi.quantity * oi.price**.

► \$UNWIND AUR LEFT JOIN — PRESERVE NULL AND EMPTY ARRAYS

MongoDB

```
db.users.aggregate([
  { $lookup: { from: "orders", localField: "userId", foreignField: "userId", as: "orders" } },
  { $unwind: { path: "$orders", preserveNullAndEmptyArrays: true } }, // Anita bachegi
  { $project: { _id: 0, name: 1, orderId: "$orders.orderId" } }
])
// { name: "Sonu", orderId: "01001" }, { name: "Sonu", orderId: "01003" },
// { name: "Priya", orderId: "01002" }, { name: "Rahul", orderId: "01004" },
// { name: "Anita" } // ← orderId missing (SQL mein NULL hota)

// Default $unwind (bina option) → Anita GAYAB = INNER JOIN behaviour
```

	\$arrayElemAt / \$first	\$unwind
Result shape	Documents ki count same	Har array element ek document
Empty array	Field missing, doc rehta hai	Doc gayab (jab tak preserve: true na ho)
Use	Many-to-one (order → user)	One-to-many flatten, ya \$group se pehle
SQL feel	LEFT JOIN, one match	JOIN jo rows multiply karta hai

COMMON MISTAKES

- Bade array pe unwind — 1 document × 10,000 items = 10,000 documents pipeline mein. Pehle \$match se kam karo.
- Unwind ke baad `_id` duplicate ho jaata hai (sab pieces ka same `_id`) — baad mein `$out`/insert karoge to duplicate key.
- `$unwind: "items"` — \$ bhoool gaye. Field path `"$items"` hona chahiye.

INTERVIEW QUESTION

Q: \$unwind kya karta hai aur \$lookup ke saath kyun use hota hai?

Answer: \$unwind array field ko tod ke har element ke liye alag document banata hai. \$lookup result array hota hai; unwind karne se SQL JOIN jaisa flat result milta hai aur aage \$group / \$sort per-element kar sakte hain. Default unwind empty/missing arrays wale documents drop karta hai (INNER jaisa); `preserveNullAndEmptyArrays: true` se LEFT jaisa.

MUST KNOW	GOOD TO KNOW	NOT NECESSARY RIGHT NOW
• \$unwind kya karta hai • <code>preserveNullAndEmptyArrays</code> • <code>\$arrayElemAt</code> vs <code>\$unwind</code> kab	• Unwind se document explosion • <code>\$first</code> shortcut	• <code>includeArrayIndex</code> option

6.5 Condition wala \$lookup (pipeline form)

Requirement: 'Har user ke sirf PAID orders jodo'. Simple \$lookup sirf equality karta hai. Extra condition ke liye **pipeline \$lookup**:

SQL (dono)

```
SELECT u.name, o.order_id, o.total
FROM users u
LEFT JOIN orders o ON o.user_id = u.user_id AND o.status = 'PAID';
```

MongoDB

```
db.users.aggregate([
  { $lookup: {
    from: "orders",
    localField: "userId", foreignField: "userId",          // MongoDB 5.0+: dono saath chalte hain
    pipeline: [
      { $match: { status: "PAID" } },
      { $project: { _id: 0, orderId: 1, total: 1 } }        // sirf zaroori fields laao
    ],
    as: "paidOrders"
  } },
  { $project: { _id: 0, name: 1, paidOrders: 1 } }
])
// Sonu: [01001], Priya: [], Rahul: [01004], Anita: []
```

MongoDB 5.0 se pehle **let + \$expr** likhna padta tha — purane code mein dikhega, samajh lena kaafi hai. Inner **\$project** important hai: bade documents ko pura laane se memory bachti hai.

MUST KNOW	GOOD TO KNOW	NOT NECESSARY RIGHT NOW
- Pipeline \$lookup exist karta hai - Extra condition ON mein	let + \$expr purana syntax	Correlated subquery optimization

6.6 Performance, indexes aur 'kab JOIN / \$lookup karna'

- JOIN column pe index:** `orders.user_id`, `order_items.product_id`. Postgres FK pe index khud nahi banata (PDF 1).
- \$lookup ka foreignField indexed ho:** `db.users.createIndex({ userId: 1 })`. Bina index ke har input document ke liye poori foreign collection scan = $N \times M$.
- \$match pehle, \$lookup baad mein** — 1 lakh orders pe lookup mat karo agar 20 hi chahiye. Filter → sort → limit → phir lookup.
- SQL databases JOIN ke liye decades se optimized hain (query planner join order aur algorithm khud chunta hai). MongoDB \$lookup kaam karta hai, lekin **heavy multi-collection joins Mongo ka strong point nahi**. Agar har API call 4-5 lookups kar rahi hai, to data model dobara socho (embed karo — Part 9).
- Sharded collections pe \$lookup ke limitations rahe hain (naye versions mein kaafi improve hua) — GOOD TO KNOW.

MongoDB

```
// Paginated order list with customer name – sahi order
db.orders.aggregate([
  { $match: { status: "PAID" } },          // 1. filter (index use)
  { $sort: { createdAt: -1 } },           // 2. sort (index use)
  { $limit: 20 },                         // 3. sirf 20 bache
  { $lookup: { from: "users", localField: "userId", foreignField: "userId",
    pipeline: [{ $project: { _id: 0, name: 1 } }], as: "user" } }, // 4. 20 lookups
  { $set: { customer: { $first: "$user.name" } } },
  { $unset: "user" }
])
```

6.7 Node.js mein JOIN — 6 tareeke

Node.js

```
// ----- pg -----
const { rows } = await pool.query(`
  SELECT o.order_id, o.total, o.status, u.name AS customer
  FROM orders o JOIN users u ON u.user_id = o.user_id
  WHERE o.user_id = $1 ORDER BY o.created_at DESC LIMIT 20`, [userId]);

// ----- mysql2 -----
const [rows2] = await pool.execute(`
  SELECT o.order_id, o.total, u.name AS customer
  FROM orders o JOIN users u ON u.user_id = o.user_id
  WHERE o.user_id = ? ORDER BY o.created_at DESC LIMIT 20`, [userId]);

// Nested JSON seedha SQL se (Postgres) — 1 query mein order + items array
const { rows: withItems } = await pool.query(`
  SELECT o.order_id, o.total,
    json_agg(json_build_object('productId', oi.product_id, 'qty', oi.quantity)) AS items
  FROM orders o JOIN order_items oi ON oi.order_id = o.order_id
  WHERE o.order_id = $1 GROUP BY o.order_id, o.total`, [orderId]);
// MySQL 8: JSON_ARRAYAGG(JSON_OBJECT('productId', oi.product_id, 'qty', oi.quantity))

// ----- MongoDB driver -----
const orders = await db.collection("orders").aggregate([
  { $match: { userId } }, { $sort: { createdAt: -1 } }, { $limit: 20 },
  { $lookup: { from: "users", localField: "userId", foreignField: "userId", as: "user" } },
  { $set: { customer: { $first: "$user.name" } } }, { $unset: "user" },
]).toArray();

// ----- Prisma -----
await prisma.order.findMany({
  where: { userId }, take: 20, orderBy: { createdAt: "desc" },
  include: { user: { select: { name: true } },
    items: { include: { product: { select: { name: true } } } } },
});

// ----- Sequelize -----
await Order.findAll({ where: { userId }, limit: 20,
  include: [{ model: User, attributes: ["name"] },
    { model: OrderItem, include: [{ model: Product, attributes: ["name"] } ] } ] });

// ----- Mongoose populate (Part 18 mein detail) -----
await Order.find({ userId }).limit(20)
  .populate({ path: "user", select: "name" }) // ref + virtual setup chahiye
  .lean();
// populate = alag queries (orders, phir users WHERE _id IN [...]) → app mein jodna.
// $lookup = DB ke andar ek aggregation. Dono ka result similar, mechanism alag.
```

► N+1 PROBLEM — PEHLI JHALAK (PART 19 MEIN POORA)

Node.js

```
// GALAT — 20 orders → 1 + 20 = 21 queries
const orders = await db.collection("orders").find().limit(20).toArray();
for (const o of orders) {
  o.user = await db.collection("users").findOne({ userId: o.userId }); // loop mein query!
}

// SAHI — 2 queries (ya ek $lookup / JOIN)
const ids = [...new Set(orders.map(o => o.userId))];
const users = await db.collection("users").find({ userId: { $in: ids } }).toArray();
const byId = new Map(users.map(u => [u.userId, u]));
orders.forEach(o => (o.user = byId.get(o.userId)));
```

COMMON MISTAKES

- Loop ke andar DB query (N+1).
- Prisma/Sequelize mein deep **include** — 4 level nesting = bahut badi query ya bahut saari queries. Sirf zaroori relations.
- Mongoose mein populate aur \$lookup ko exactly same maanna — populate application-level hai, multiple round trips.

INTERVIEW QUESTION

Q: MongoDB mein JOIN kaise karte ho, aur kab JOIN avoid karna chahiye?

Answer: \$lookup (aggregation) ya Mongoose populate (application-level). Avoid karna chahiye jab same data hamesha saath padha jaata hai — tab embed karo, ek read mein sab. Agar har request multiple \$lookup kar rahi hai, to ya to model galat hai ya relational DB better fit hai. Lookup karo to foreignField pe index aur pehle \$match/\$limit.

MUST KNOW	GOOD TO KNOW	NOT NECESSARY RIGHT NOW
• JOIN/lookup ke columns pe index • \$match → \$limit → \$lookup order • N+1 problem • populate ≠ \$lookup (mechanism)	• json_agg / JSON_ARRAYAGG • Prisma include / Sequelize include	• Sharded \$lookup internals

Part 6 — Memory Table

SQL	MongoDB	Yaad rakho
JOIN t ON a = b	\$lookup { from, localField, foreignField, as }	Result array
INNER JOIN	\$lookup + \$match { arr: { \$ne: [] } }	Ya plain \$unwind
LEFT JOIN	\$lookup (default)	No match → []
LEFT JOIN ... WHERE r.id IS NULL	\$match { arr: { \$size: 0 } }	Anti-join
Many-to-one column (u.name)	\$arrayElemAt / \$first	Array → object
Rows multiply (order_items)	\$unwind	Array → multiple docs
LEFT + flatten	\$unwind + preserveNullAndEmptyArrays	
JOIN ... ON a = b AND x = 'P'	\$lookup with pipeline	Extra condition
col AS alias	\$project { alias: '\$f' }	
json_agg / JSON_ARRAYAGG	(natural — embedded arrays)	Nested response
ORM include	Mongoose populate	App-level, multiple queries

PART 7 — GROUP BY vs \$group

7.1 GROUP BY kya hai?

► CONCEPT

Rows ko kisi field ki value ke hisaab se **groups (dheriyon)** mein baantna, aur har group ke liye **ek summary row** nikalna — count, sum, average, min, max.

► REAL-WORLD EXAMPLE

Dukaan ka hisaab: saari parchiyan table pe hain. 'Har customer ne kitna kharida?' — parchiyon ko customer ke naam se alag dheriyon mein rakho, phir har dheri ka total jodo. Dheri = group, jodna = **SUM**.

Output

orders:		GROUP BY user_id:
01001 1103 1500		1103 → [1500, 280] → SUM = 1780
01002 1104 900	→	1104 → [900] → SUM = 900
01003 1103 280		1105 → [1000] → SUM = 1000
01004 1105 1000		

► REQUIREMENT: 'FIND TOTAL SALES PER USER'

SQL (dono)

```
SELECT
  user_id,
  SUM(total) AS total_sales
FROM orders
GROUP BY user_id
ORDER BY user_id;
```

MongoDB

```
db.orders.aggregate([
  {
    $group: {
      _id: "$userId",           // kis field se group karna hai
      totalSales: { $sum: "$total" } // har group mein kya calculate karna hai
    },
  },
  { $sort: { _id: 1 } }
])
```

Output

SQL:	MongoDB:
user_id total_sales	{ _id: "1103", totalSales: 1780 }
-----+-----	{ _id: "1104", totalSales: 900 }
1103 1780	{ _id: "1105", totalSales: 1000 }
1104 900	
1105 1000	

► LINE-BY-LINE

- **GROUP BY user_id** — same user_id wali rows ek group. Output mein **har group ki ek row**.
- **SUM(total)** — **aggregate function**: group ki saari rows ka total jodo.

- **SELECT user_id, ...** — SELECT mein sirf wahi columns jo GROUP BY mein hain, ya aggregate functions.
- **_id: "\$userId"** — Mongo mein group key **hamesha _id naam** se aati hai. **\$** lagana zaroori (field ki value).
- **totalSales: { \$sum: "\$total" }** — naya field naam : accumulator. **\$sum = SUM**.
- Mongo \$group ke output mein sirf **_id + accumulators** hote hain — baaki fields (name, city) **gayab**. Chahiye to accumulator se laao (**\$first**) ya baad mein \$lookup.

Dhyan do: ye **saare statuses** ka total hai — CANCELLED O1003 (280) bhi juda. Real 'sales' mein sirf PAID chahiye → 7.3.

MUST KNOW	GOOD TO KNOW	NOT NECESSARY RIGHT NOW
• GROUP BY ↔ \$group • _id = group key • SELECT mein sirf group columns + aggregates	• \$group output mein baaki fields nahi aate	• Hash vs sort aggregation internals

7.2 COUNT, SUM, AVG, MIN, MAX — side by side

SQL (dono)		
<pre>SELECT user_id, COUNT(*) AS orders, SUM(total) AS total_spent, ROUND(AVG(total),2) AS avg_order, MIN(total) AS smallest, MAX(total) AS biggest FROM orders GROUP BY user_id ORDER BY total_spent DESC;</pre>		
MongoDB		
<pre>db.orders.aggregate([{ \$group: { _id: "\$userId", orders: { \$sum: 1 }, // COUNT(*) → har doc ke liye 1 jodo totalSpent: { \$sum: "\$total" }, avgOrder: { \$avg: "\$total" }, smallest: { \$min: "\$total" }, biggest: { \$max: "\$total" } } }, { \$set: { avgOrder: { \$round: ["\$avgOrder", 2] } } }, { \$sort: { totalSpent: -1 } }])</pre>		
Output		
<pre>user_id orders total_spent avg_order smallest biggest -----+-----+-----+-----+-----+----- 1103 2 1780 890.00 280 1500 1105 1 1000 1000.00 1000 1000 1104 1 900 900.00 900 900</pre>		
Kaam	SQL	MongoDB accumulator
Count rows	COUNT(*)	{ \$sum: 1 } (ya { \$count: {} } — 5.0+)

Kaam	SQL	MongoDB accumulator
Sum	<code>SUM(col)</code>	<code>{ \$sum: "\$col" }</code>
Average	<code>AVG(col)</code>	<code>{ \$avg: "\$col" }</code>
Min / Max	<code>MIN(col) / MAX(col)</code>	<code>{ \$min: "\$col" } / { \$max: ... }</code>
Pehli/aakhri value	— (window functions)	<code>{ \$first: "\$col" } / \$last</code> (sort ke baad)
List bana do	PG: <code>array_agg(col)</code> , MySQL: <code>JSON_ARRAYAGG(col)</code>	<code>{ \$push: "\$col" }</code>
Unique list	PG: <code>array_agg(DISTINCT col)</code>	<code>{ \$addToSet: "\$col" }</code>
String join	PG: <code>string_agg(col, ',')</code> , MySQL: <code>GROUP_CONCAT(col)</code>	<code>\$push + \$reduce</code> / app mein join

► POORI TABLE KA EK HI SUMMARY (GROUP BY KE BINA)

SQL (dono)

```
SELECT COUNT(*) AS orders, SUM(total) AS revenue, AVG(total) AS avg_order FROM orders;
-- orders | revenue | avg_order
--      4 |    3680 |    920.00
```

MongoDB

```
db.orders.aggregate([
  { $group: { _id: null, orders: { $sum: 1 },
              revenue: { $sum: "$total" }, avgOrder: { $avg: "$total" } } }
])
// { _id: null, orders: 4, revenue: 3680, avgOrder: 920 }
```

`_id: null` = sab documents ek hi group mein. Sirf count chahiye to simple:
`db.orders.countDocuments({ status: "PAID" })`.

► COUNT(*) VS COUNT(COLUMN) — NULL KA KHEL

SQL (dono)

```
SELECT COUNT(*) AS all_users, COUNT(email) AS with_email FROM users;
-- all_users | with_email
--      4 |      3      ← Rahul ka email NULL, COUNT(email) ne skip kiya
```

MongoDB

```
db.users.aggregate([
  { $group: {
    _id: null,
    allUsers: { $sum: 1 },
    withEmail: { $sum: { $cond: [ { $ifNull: ["$email", false] }, 1, 0 ] } }
  } }
])
// { _id: null, allUsers: 4, withEmail: 3 }
// Simple tareeka: db.users.countDocuments({ email: { $ne: null } }) → 3
```

Aggregate functions (`SUM`, `AVG`, `MIN`, `MAX`, `COUNT(col)`) **NULL ko ignore** karte hain. `AVG` NULL ko zero nahi maanta — NULL wali rows average ke denominator mein bhi nahi ginti. Mongo `$avg` / `$sum` bhi non-numeric/missing values ignore karte hain.

MUST KNOW	GOOD TO KNOW	NOT NECESSARY RIGHT NOW
5 aggregates dono syntax mein \$sum: 1 = COUNT _id: null = poori collection - COUNT(*) vs COUNT(col)	\$push/\$addToSet ↔ array_agg/JSON_ARRAYAGG - GROUP_CONCAT / string_agg \$first/\$last	Statistical aggregates (stddev, percentile)

7.3 WHERE vs HAVING → \$match pehle vs \$match baad mein

► CONCEPT

	Kab filter karta hai	Kis cheez pe
WHERE	Grouping se pehle	Individual rows (status = 'PAID')
HAVING	Grouping ke baad	Group ke result (SUM(total) > 1000)

► REQUIREMENT: 'SIRF PAID ORDERS KI SALES PER USER, AUR SIRF WO USERS JINKI SALES 1000 YA ZYADA'

SQL (dono)

```
SELECT user_id, COUNT(*) AS paid_orders, SUM(total) AS sales
FROM orders
WHERE status = 'PAID'           -- 1. rows filter (CANCELLED/PENDING hatao)
GROUP BY user_id               -- 2. group
HAVING SUM(total) >= 1000      -- 3. groups filter
ORDER BY sales DESC;
```

MongoDB

```
db.orders.aggregate([
  { $match: { status: "PAID" } },           // WHERE
  { $group: { _id: "$userId", paidOrders: { $sum: 1 }, sales: { $sum: "$total" } } },
  { $match: { sales: { $gte: 1000 } } },    // HAVING
  { $sort: { sales: -1 } }
])
```

Output

```
user_id | paid_orders | sales
-----+-----+-----
1103    |           1 | 1500
1105    |           1 | 1000
(Priya ka 01002 PENDING hai → WHERE ne hataya → group hi nahi bana)
```

► SQL EXECUTION ORDER — YE SAMJHO TO GROUP BY KABHI CONFUSE NAHI KAREGA

Output

Likhte hain: SELECT → FROM → WHERE → GROUP BY → HAVING → ORDER BY → LIMIT
 Chalta hai: FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY → LIMIT

Mongo pipeline mein jo likha, wahi order mein chalta hai:
 \$match → \$group → \$match → \$project → \$sort → \$limit

Isliye SQL mein **WHERE sales > 1000** error deta hai (sales alias abhi bana hi nahi, SELECT baad mein chalta hai). PostgreSQL mein HAVING mein bhi alias allowed nahi — **HAVING SUM(total) >= 1000** likhna padta hai. MySQL HAVING mein alias allow karta hai (**HAVING sales >= 1000**).

COMMON MISTAKES

- `WHERE SUM(total) > 1000` — error. Aggregate filter HAVING mein.
- HAVING mein aisi condition jo WHERE mein ja sakti thi (`HAVING status = 'PAID'`) — pehle saari rows group hongi, phir filter; slow. Row-level filter hamesha WHERE / pehla `$match`.
- Mongo mein `$match` ko `$group` ke baad rakha jab wo pehle ho sakta tha — index use nahi hoga, saare docs group honge.

INTERVIEW QUESTION

Q: WHERE aur HAVING mein kya difference hai? MongoDB mein kaise?

Answer: WHERE grouping se pehle individual rows filter karta hai, HAVING grouping ke baad groups filter karta hai (aggregate values pe). MongoDB mein dono `$match` hi hain — `$group` se pehle wala WHERE, baad wala HAVING. Performance ke liye row filters hamesha pehle, kyunki pehla `$match` index use kar sakta hai.

MUST KNOW	GOOD TO KNOW	NOT NECESSARY RIGHT NOW
• WHERE vs HAVING • <code>\$match</code> before/after <code>\$group</code> • SQL logical execution order	• PG vs MySQL HAVING alias	• —

7.4 Multiple fields se group, dates se group, JOIN ke saath group

► A) USER + STATUS DONO SE

SQL (dono)

```
SELECT user_id, status, COUNT(*) AS n, SUM(total) AS amt
FROM orders GROUP BY user_id, status ORDER BY user_id, status;
```

MongoDB

```
db.orders.aggregate([
  { $group: { _id: { userId: "$userId", status: "$status" },
    n: { $sum: 1 }, amt: { $sum: "$total" } } },
  { $sort: { "_id.userId": 1, "_id.status": 1 } }
])
// { _id: { userId: "1103", status: "CANCELLED" }, n: 1, amt: 280 }
// { _id: { userId: "1103", status: "PAID" }, n: 1, amt: 1500 }
// { _id: { userId: "1104", status: "PENDING" }, n: 1, amt: 900 }
// { _id: { userId: "1105", status: "PAID" }, n: 1, amt: 1000 }
```

Composite group key Mongo mein **object** ban jaati hai. Flat chahiye to baad mein `$project: { userId: "$_id.userId", ... }`.

► B) DATE SE GROUP — DAILY SALES REPORT

SQL (dono)

```
SELECT DATE(created_at) AS day, COUNT(*) AS orders, SUM(total) AS revenue
FROM orders WHERE status = 'PAID'
GROUP BY DATE(created_at) ORDER BY day;
-- PostgreSQL mein month: DATE_TRUNC('month', created_at)
-- MySQL mein month:     DATE_FORMAT(created_at, '%Y-%m')
```

MongoDB

```
db.orders.aggregate([
  { $match: { status: "PAID" } },
  { $group: {
    _id: { $dateToString: { format: "%Y-%m-%d", date: "$createdAt", timezone: "Asia/Kolkata" } },
    orders: { $sum: 1 }, revenue: { $sum: "$total" }
  } },
  { $sort: { _id: 1 } }
])
// { _id: "2026-09-01", orders: 1, revenue: 1500 }
// { _id: "2026-09-08", orders: 1, revenue: 1000 } ← 18:45 UTC = 00:15 IST agle din!
// Month ke liye: { $dateTrunc: { date: "$createdAt", unit: "month" } } (5.0+)
```

Timezone trap: DB mein time UTC mein hota hai. O1004 UTC mein 7 Sept hai, lekin India mein 8 Sept. Daily report kis timezone mein chahiye, ye hamesha explicitly batao (Postgres: `created_at AT TIME ZONE 'Asia/Kolkata'`).

► C) JOIN + GROUP BY — PRODUCT-WISE UNITS AUR REVENUE

SQL (dono)

```
SELECT p.product_id, p.name,
       SUM(oi.quantity)      AS units,
       SUM(oi.quantity * oi.price) AS revenue
FROM order_items oi
JOIN orders o ON o.order_id = oi.order_id
JOIN products p ON p.product_id = oi.product_id
WHERE o.status = 'PAID'
GROUP BY p.product_id, p.name
ORDER BY revenue DESC;
-- P101 | Steel Pipe | 5 | 2500
```

MongoDB

```
db.orders.aggregate([
  { $match: { status: "PAID" } },
  { $unwind: "$items" },
  { $group: {
    _id: "$items.productId",
    units: { $sum: "$items.quantity" },
    revenue: { $sum: { $multiply: ["$items.quantity", "$items.price"] } }
  } },
  { $lookup: { from: "products", localField: "_id", foreignField: "productId", as: "p" } },
  { $project: { _id: 0, productId: "$_id", name: { $first: "$p.name" }, units: 1, revenue: 1 } },
  { $sort: { revenue: -1 } }
])
// { productId: "P101", name: "Steel Pipe", units: 5, revenue: 2500 }
```

Notice: Mongo mein **pehle group, phir lookup** — 1000 order lines ki jagah sirf unique products ke liye lookup. SQL mein planner ye khud decide karta hai; Mongo mein order tum decide karte ho.

► D) CONDITIONAL AGGREGATION — EK QUERY MEIN STATUS-WISE COUNTS

PostgreSQL

```
SELECT
  COUNT(*) FILTER (WHERE status = 'PAID')      AS paid,
  COUNT(*) FILTER (WHERE status = 'PENDING')   AS pending,
  COUNT(*) FILTER (WHERE status = 'CANCELLED') AS cancelled
FROM orders;
-- paid 2, pending 1, cancelled 1
```

MySQL

```
SELECT
  SUM(status = 'PAID')      AS paid,      -- MySQL mein boolean = 1/0, isliye SUM chalta hai
  SUM(status = 'PENDING')  AS pending,
  SUM(status = 'CANCELLED') AS cancelled
FROM orders;
-- Portable (dono DB): SUM(CASE WHEN status = 'PAID' THEN 1 ELSE 0 END)
```

MongoDB

```
db.orders.aggregate([
  { $group: { _id: null,
    paid:      { $sum: { $cond: [{ $eq: ["$status", "PAID"] }, 1, 0] } },
    pending:   { $sum: { $cond: [{ $eq: ["$status", "PENDING"] }, 1, 0] } },
    cancelled: { $sum: { $cond: [{ $eq: ["$status", "CANCELLED"] }, 1, 0] } } } }
])
// Ya simple: { $group: { _id: "$status", n: { $sum: 1 } } } → 3 docs
```

7.5 'must appear in GROUP BY' error

SQL (dono)

```
SELECT user_id, status, SUM(total) FROM orders GROUP BY user_id;

PostgreSQL: ERROR: column "orders.status" must appear in the GROUP BY clause
             or be used in an aggregate function
MySQL 5.7+: ERROR 1055: ... this is incompatible with sql_mode=only_full_group_by
```

Logic: user 1103 ke group mein do statuses hain (PAID, CANCELLED) — DB kaunsa dikhaye? Isliye error. Purana MySQL (ya **ONLY_FULL_GROUP_BY** off) **koi bhi random value** dikha deta tha — silent bug. Fix: column ko GROUP BY mein daalo, ya aggregate lagao (**MAX(status)**), ya sahi mein chahiye kya ye socho.

Exception (dono DB): agar GROUP BY **primary key** pe hai, to usi table ke baaki columns allowed hain (functional dependency) — **GROUP BY u.user_id** karke **u.name** select kar sakte ho.

7.6 Node.js mein GROUP BY

Node.js

```
// ----- pg -----
const { rows } = await pool.query(`
  SELECT user_id, COUNT(*)::int AS orders, SUM(total) AS sales
  FROM orders WHERE status = $1 GROUP BY user_id ORDER BY sales DESC`, ["PAID"]);
// GOTCHA: pg COUNT(*) ko bigint → STRING ("2") return karta hai, NUMERIC bhi string ("1500.00").
// JS number precision bachane ke liye. Fix: ::int cast, ya pg.types.setTypeParser.

// ----- mysql2 -----
const [rows2] = await pool.execute(
  `SELECT user_id, COUNT(*) AS orders, SUM(total) AS sales
  FROM orders WHERE status = ? GROUP BY user_id`,
  ["PAID"]);
// GOTCHA: DECIMAL SUM string mein aata hai ("1500.00") – decimalNumbers: true option ya parseFloat.

// ----- MongoDB driver -----
const stats = await db.collection("orders").aggregate([
  { $match: { status: "PAID" } },
  { $group: { _id: "$userId", orders: { $sum: 1 }, sales: { $sum: "$total" } } },
  { $sort: { sales: -1 } },
]).toArray();

// ----- Prisma -----
await prisma.order.groupBy({
  by: ["userId"],
  where: { status: "PAID" },
  _count: { _all: true },
  _sum: { total: true },
  having: { total: { _sum: { gte: 1000 } } },
  orderBy: { _sum: { total: "desc" } },
});
// Complex report → prisma.$queryRaw`...` (tagged template = safe parameters)

// ----- Sequelize -----
await Order.findAll({
  attributes: ["userId", [fn("COUNT", col("*")), "orders"], [fn("SUM", col("total")), "sales"]],
  where: { status: "PAID" }, group: ["userId"], raw: true,
});

// ----- Mongoose -----
await Order.aggregate([
  { $match: { status: "PAID" } },
  { $group: { _id: "$userId", sales: { $sum: "$total" } } },
]);
// GOTCHA: Model.aggregate() schema casting NAHI karta – ObjectId string se match nahi hoga,
// khud new mongoose.Types.ObjectId(id) karo.
```

COMMON MISTAKES

- pg se aaye "2" (string count) pe `count + 1` → "21". Types check karo.
- Har page load pe bade table pe GROUP BY report chalana — heavy. Pre-computed summary / cache / materialized view (Part 19).
- Money SUM ko JS float mein convert karke aur calculations — precision loss. Paise (integer) ya decimal library.

INTERVIEW QUESTION

Q: Har customer ka total PAID amount aur order count nikalo, sirf top 5 — SQL aur MongoDB dono mein likho.

Answer: SQL: `SELECT user_id, COUNT(*), SUM(total) AS s FROM orders WHERE status='PAID' GROUP BY user_id ORDER BY s DESC LIMIT 5.` Mongo: `[{$match:{status:'PAID'}}, {$group:{_id:'$userId', n:{$sum:1}, s:{$sum:'$total'}}}, {$sort:{s:-1}}, {$limit:5}]`. Status + userId pe index filter ko fast karega.

MUST KNOW	GOOD TO KNOW	NOT NECESSARY RIGHT NOW
- GROUP BY multiple columns / object_id - must appear in GROUP BY — kyun - pg bigint/numeric string gotcha - Mongo: group pehle, lookup baad	- Date grouping + timezone - FILTER / SUM(CASE) / \$cond - Prisma groupBy - Mongoose aggregate no-cast	- GROUPING SETS / ROLLUP / CUBE - Window functions (baad mein, optional)

Part 7 — Memory Table

SQL	MongoDB	Note
GROUP BY col	<code>\$group: { _id: "\$col" }</code>	Key ka naam _id
GROUP BY a, b	<code>_id: { a: "\$a", b: "\$b" }</code>	Object key
(no GROUP BY) aggregate	<code>_id: null</code>	Ek group
COUNT(*)	<code>{ \$sum: 1 }</code>	countDocuments bhi
SUM / AVG / MIN / MAX	<code>\$sum / \$avg / \$min / \$max</code>	NULL ignore
array_agg / JSON_ARRAYAGG	<code>\$push / \$addToSet</code>	
WHERE	<code>\$match</code> (group se pehle)	Index use
HAVING	<code>\$match</code> (group ke baad)	
DATE_TRUNC / DATE_FORMAT	<code>\$dateTrunc / \$dateToString</code>	Timezone!
SUM(CASE WHEN ...)	<code>\$sum + \$cond</code>	Conditional count

PART 8 — Aggregation Pipeline

8.1 Pipeline kya hai?

► CONCEPT

Aggregation pipeline = **factory ki conveyor belt**. Documents ek taraf se andar aate hain, har **stage** unpe ek kaam karta hai (filter, jodna, todna, group, shape badalna, sort), aur agla stage pichhle stage ka output leta hai.

Output

```
orders collection
|
▼
[$match] → sirf kaam ke documents (WHERE)
▼
[$lookup] → doosri collection jodo (JOIN)
▼
[$unwind] → array ko rows mein todo (rows expand)
▼
[$group] → summary banao (GROUP BY)
▼
[$project] → shape / fields decide (SELECT)
▼
[$sort] → order (ORDER BY)
▼
[$limit] → top N (LIMIT)
▼
result
```

► WHY?

- **find()** sirf filter + projection + sort kar sakta hai. Join, group, calculations, reshape — sab pipeline mein.
- Reports, dashboards, API response shaping — sab DB ke andar, Node.js mein data laake loop lagane ki zaroorat nahi.
- SQL mein ek query mein sab clauses hote hain aur **DB order decide karta hai**. Mongo pipeline mein **tum order decide karte ho** — power bhi, zimmedari bhi.

► MEMORY MODEL

Mongo stage	SQL	Kaam
\$match	WHERE / HAVING	Filter
\$lookup	JOIN	Doosri collection jodo
\$unwind	(JOIN jo rows multiply kare)	Array → multiple docs
\$group	GROUP BY + aggregates	Summary
\$project	SELECT	Fields choose / rename / compute
\$addFields / \$set	SELECT *, extra_col	Naya field, baaki sab rakho
\$unset	(column mat select karo)	Field hatao
\$sort	ORDER BY	Order
\$skip / \$limit	OFFSET / LIMIT	Pagination

Mongo stage	SQL	Kaam
<code>\$count</code>	<code>SELECT COUNT(*)</code>	Kitne docs bache
<code>\$facet</code>	Kai queries ek saath (UNION ALL / alag queries)	Multiple sub-pipelines, ek result

8.2 Complete Example — step by step

► REQUIREMENT

'User 1103 ke PAID orders nikalo, user aur product information jodo, har line ka total calculate karo, aur price ke hisaab se sort karo.'

Sonu ka sirf ek PAID order hai (O1001), isliye har stage ka effect clearly dikhane ke liye maan lo O1001 mein ek aur item bhi hai: **P103 Elbow Joint × 4 @ 80**. (Practice ke liye ye update chala lo.)

MongoDB

```
db.orders.updateOne(
  { orderId: "O1001" },
  { $push: { items: { productId: "P103", quantity: 4, price: 80 } }, $set: { total: 1820 } }
)
```

SQL (dono)

```
INSERT INTO order_items (order_id, product_id, quantity, price) VALUES ('O1001', 'P103', 4, 80);
UPDATE orders SET total = 1820 WHERE order_id = 'O1001';
```

► POORI MONGODB PIPELINE

MongoDB

```
db.orders.aggregate([
  // 1. WHERE
  { $match: { userId: "1103", status: "PAID" } },

  // 2. JOIN users
  { $lookup: {
    from: "users", localField: "userId", foreignField: "userId",
    pipeline: [ { $project: { _id: 0, name: 1, city: 1 } } ],
    as: "user"
  } },
  { $set: { user: { $first: "$user" } } },

  // 3. Items ko rows mein todo
  { $unwind: "$items" },

  // 4. JOIN products
  { $lookup: {
    from: "products", localField: "items.productId", foreignField: "productId",
    pipeline: [ { $project: { _id: 0, name: 1, categoryId: 1 } } ],
    as: "product"
  } },
  { $unwind: "$product" },

  // 5. SELECT + calculation
  { $project: {
    _id: 0,
    orderId: 1,
    customer: "$user.name",
    city: "$user.city",
    productId: "$items.productId",
    product: "$product.name",
    quantity: "$items.quantity",
    price: "$items.price",
    lineTotal: { $multiply: ["$items.quantity", "$items.price"] }
  } },

  // 6. ORDER BY
  { $sort: { price: -1 } }
])
```

► STAGE-BY-STAGE — PIPELINE KE ANDAR KYA HO RAHA HAI

Output

```
STAGE 1 $match → 4 orders mein se 1 bacha
{ orderId: "01001", userId: "1103", status: "PAID", total: 1820,
  items: [ { productId: "P101", quantity: 3, price: 500 },
    { productId: "P103", quantity: 4, price: 80 } ], ... }

STAGE 2 $lookup users + $set $first → user object juda
{ orderId: "01001", ..., items: [ ...2 items... ],
  user: { name: "Sonu", city: "Mumbai" } }

STAGE 3 $unwind items → 1 doc → 2 docs
{ orderId: "01001", items: { productId: "P101", quantity: 3, price: 500 }, user: {...} }
{ orderId: "01001", items: { productId: "P103", quantity: 4, price: 80 }, user: {...} }

STAGE 4 $lookup products + $unwind → product juda
{ ..., items: { productId: "P101", ... }, product: { name: "Steel Pipe", categoryId: "C10" } }
{ ..., items: { productId: "P103", ... }, product: { name: "Elbow Joint", categoryId: "C20" } }

STAGE 5 $project → flat, clean shape + lineTotal
STAGE 6 $sort price desc
```

Output

```
[
  { orderId: "01001", customer: "Sonu", city: "Mumbai", productId: "P101",
    product: "Steel Pipe", quantity: 3, price: 500, lineTotal: 1500 },
  { orderId: "01001", customer: "Sonu", city: "Mumbai", productId: "P103",
    product: "Elbow Joint", quantity: 4, price: 80, lineTotal: 320 }
]
```

► EQUIVALENT SQL

SQL (dono)

```
SELECT
  o.order_id,
  u.name           AS customer,
  u.city,
  oi.product_id,
  p.name           AS product,
  oi.quantity,
  oi.price,
  oi.quantity * oi.price AS line_total
FROM orders o
JOIN users      u ON u.user_id   = o.user_id      -- $lookup users
JOIN order_items oi ON oi.order_id = o.order_id    -- $unwind items
JOIN products   p ON p.product_id = oi.product_id  -- $lookup products
WHERE o.user_id = '1103'                          -- $match
  AND o.status  = 'PAID'
ORDER BY oi.price DESC;                            -- $sort
```

Output

order_id	customer	city	product_id	product	quantity	price	line_total
01001	Sonu	Mumbai	P101	Steel Pipe	3	500	1500
01001	Sonu	Mumbai	P103	Elbow Joint	4	80	320

► LINE-BY-LINE (IMPORTANT CHEEZEIN)

- **Stage 1 \$match sabse pehle** — sirf yahi stage (aur pehla \$sort) collection ka **index** use kar sakta hai. Baad ke stages memory mein chal rahe data pe kaam karte hain.
- \$lookup ke andar **pipeline: [{ \$project }]** — poora user document (address, email...) nahi laaye, sirf name, city. Memory bachi.
- **\$set: { user: { \$first: "\$user" } }** — order ka ek hi user hota hai, array → object.
- **\$unwind: "\$items"** — SQL ke **JOIN order_items** jaisa. Iske baad har doc ek item hai, isliye product lookup per-item hua.
- **\$project** — final API shape. **"\$items.quantity"** nested value, **\$multiply** computed column.
- SQL mein JOIN order tumne likha, lekin DB ka query planner **khud best order chunta hai**. Mongo mein jo likha wahi chalega — isliye stage order soch ke likho.

8.3 Doosra Example — Sales report with \$group, \$limit, \$count, \$facet

► REQUIREMENT (ADMIN DASHBOARD)

'Top selling products (PAID orders), har product ka units + revenue, top 3'. Aur ek hi API call mein dashboard ke 3 widgets.

MongoDB

```
db.orders.aggregate([
  { $match: { status: "PAID" } },           // 01001, 01004
  { $unwind: "$items" },                   // 3 item lines
  { $group: {
    _id: "$items.productId",
    units: { $sum: "$items.quantity" },
    revenue: { $sum: { $multiply: ["$items.quantity", "$items.price"] } },
    orders: { $addToSet: "$orderId" }
  } },
  { $lookup: { from: "products", localField: "_id", foreignField: "productId",
    pipeline: [{ $project: { _id: 0, name: 1 } }], as: "p" } },
  { $project: { _id: 0, productId: "$_id", name: { $first: "$p.name" },
    units: 1, revenue: 1, orderCount: { $size: "$orders" } } },
  { $sort: { revenue: -1 } },
  { $limit: 3 }
])
```

Output

```
[
  { productId: "P101", name: "Steel Pipe", units: 5, revenue: 2500, orderCount: 2 },
  { productId: "P103", name: "Elbow Joint", units: 4, revenue: 320, orderCount: 1 }
]
```

SQL (dono)

```
SELECT p.product_id, p.name,
       SUM(oi.quantity) AS units,
       SUM(oi.quantity * oi.price) AS revenue,
       COUNT(DISTINCT oi.order_id) AS order_count
FROM orders o
JOIN order_items oi ON oi.order_id = o.order_id
JOIN products p ON p.product_id = oi.product_id
WHERE o.status = 'PAID'
GROUP BY p.product_id, p.name
ORDER BY revenue DESC
LIMIT 3;
```

\$addToSet + \$size = COUNT(DISTINCT ...). Chhote groups ke liye theek; bahut bade groups pe array memory kha sakta hai.

► \$COUNT

MongoDB

```
db.orders.aggregate([ { $match: { status: "PAID" } }, { $count: "paidOrders" } ])
// [ { paidOrders: 2 } ] ← SELECT COUNT(*) AS paid_orders FROM orders WHERE status = 'PAID';
```

► \$FACET — EK QUERY, KAI RESULTS (DASHBOARD / SEARCH PAGE)

MongoDB

```
db.orders.aggregate([
  { $match: { createdAt: { $gte: ISODate("2026-09-01") } } },      // common filter PEHLE (index)
  { $facet: {
    summary: [
      { $group: { _id: null, totalOrders: { $sum: 1 },
        paidRevenue: { $sum: { $cond: [{ $eq: ["$status", "PAID"] }, "$total", 0] } } } },
      { $project: { _id: 0 } }
    ],
    byStatus: [
      { $group: { _id: "$status", count: { $sum: 1 }, amount: { $sum: "$total" } } },
      { $sort: { count: -1 } }
    ],
    topCustomers: [
      { $match: { status: "PAID" } },
      { $group: { _id: "$userId", spent: { $sum: "$total" } } },
      { $sort: { spent: -1 } },
      { $limit: 3 }
    ]
  } }
])
```

Output

```
[{
  summary:      [ { totalOrders: 4, paidRevenue: 2820 } ],
  byStatus:     [ { _id: "PAID", count: 2, amount: 2820 },
                  { _id: "PENDING", count: 1, amount: 900 },
                  { _id: "CANCELLED", count: 1, amount: 280 } ],
  topCustomers: [ { _id: "1103", spent: 1820 }, { _id: "1105", spent: 1000 } ]
}]
(01001 ab 1820 ka hai – 8.2 wale update ki wajah se)
```

► PAGINATION + TOTAL COUNT EK HI CALL MEIN (COMMON \$FACET USE)

MongoDB

```
db.products.aggregate([
  { $match: { categoryId: "C10" } },
  { $facet: {
    data: [ { $sort: { price: -1 } }, { $skip: 0 }, { $limit: 2 } ],
    total: [ { $count: "n" } ]
  } }
])
// [{ data: [ {Copper Pipe...}, {Steel Pipe...} ], total: [ { n: 3 } ] }]
```

► SQL MEIN \$FACET KA EQUIVALENT

SQL (dono)

```
-- Option 1: alag queries, Node.js mein Promise.all se parallel (sabse common, simple)
-- Option 2: pagination + count ek query mein – window function
SELECT product_id, name, price, COUNT(*) OVER () AS total
FROM products WHERE category_id = 'C10'
ORDER BY price DESC LIMIT 2;
-- P104 | Copper Pipe | 900 | 3
-- P101 | Steel Pipe | 500 | 3
-- (PostgreSQL + MySQL 8.0 dono. OVER () = LIMIT se pehle ke saare rows pe count)
```

Node.js

```
const [summary, byStatus, top] = await Promise.all([
  pool.query(`SELECT COUNT(*)::int AS total_orders,
    SUM(total) FILTER (WHERE status='PAID') AS paid_revenue FROM orders`),
  pool.query("SELECT status, COUNT(*)::int AS count, SUM(total) AS amount FROM orders GROUP BY status"),
  pool.query(`SELECT user_id, SUM(total) AS spent FROM orders WHERE status='PAID'
    GROUP BY user_id ORDER BY spent DESC LIMIT 3`),
]);
```

\$facet ke rules

- \$facet ka **output ek hi document** hai — 16 MB limit lagti hai. Bade result sets \$facet mein mat daalo.
- \$facet ke andar ke stages **index use nahi karte**. Common filter hamesha \$facet se **pehle** \$match mein.
- Har sub-pipeline same input pe alag se chalti hai — 5 facets = input 5 baar process.

8.4 SQL mein 'pipeline' — CTE (WITH clause)

Mongo pipeline ko step-by-step padhna aasaan lagta hai. SQL mein yahi readability **CTE** (Common Table Expression) deta hai — har step ek naam wali temporary result. PostgreSQL + MySQL 8.0+ dono mein.

SQL (dono)

```
WITH paid_orders AS (
  -- $match
  SELECT order_id, user_id FROM orders WHERE status = 'PAID'
),
lines AS (
  -- $unwind + $lookup
  SELECT po.user_id, oi.product_id, oi.quantity * oi.price AS amount
  FROM paid_orders po JOIN order_items oi ON oi.order_id = po.order_id
),
per_user AS (
  -- $group
  SELECT user_id, SUM(amount) AS spent, COUNT(DISTINCT product_id) AS products
  FROM lines GROUP BY user_id
)
SELECT u.name, pu.spent, pu.products
FROM per_user pu JOIN users u ON u.user_id = pu.user_id
ORDER BY pu.spent DESC;
-- -- $sort
-- Sonu | 1820 | 2
-- Rahul | 1000 | 1
```

8.5 Pipeline Performance Rules (interview + production)

Rule	Kyun
\$match sabse pehle	Index use hota hai; aage ke stages ko kam documents milte hain
\$sort + \$limit jaldi (agar possible)	Pehle stage mein \$sort index se; \$sort ke turant baad \$limit ho to Mongo sirf top-N memory mein rakhta hai
\$project / lookup pipeline mein fields kam	Chhote documents = kam memory, fast
\$lookup se pehle \$limit	20 docs pe lookup vs 1 lakh pe
foreignField pe index	Warna har doc ke liye foreign collection scan
\$group se pehle \$unwind soch ke	Bade arrays = document explosion

Rule	Kyun
100 MB per stage memory limit	\$group/\$sort bade data pe; 6.0+ mein zaroorat pe disk use (allowDiskUse) — slow
explain()	<code>db.orders.explain("executionStats").aggregate([...])</code> — Part 11

MongoDB kuch optimizations khud karta hai (jaise \$match ko \$project se pehle move karna jab safe ho), lekin uspe depend mat karo — sahi order khud likho.

8.6 Node.js mein aggregation

Node.js

```
// ----- Repository function (MongoDB driver) -----
async function getUserPaidOrderLines(db, userId) {
  return db.collection("orders").aggregate([
    { $match: { userId, status: "PAID" } },
    { $lookup: { from: "users", localField: "userId", foreignField: "userId",
      pipeline: [{ $project: { _id: 0, name: 1, city: 1 } }], as: "user" } },
    { $set: { user: { $first: "$user" } } },
    { $unwind: "$items" },
    { $lookup: { from: "products", localField: "items.productId", foreignField: "productId",
      pipeline: [{ $project: { _id: 0, name: 1 } }], as: "product" } },
    { $unwind: "$product" },
    { $project: { _id: 0, orderId: 1, customer: "$user.name", product: "$product.name",
      quantity: "$items.quantity", price: "$items.price",
      lineTotal: { $multiply: ["$items.quantity", "$items.price"] } } },
    { $sort: { price: -1 } },
  ], { maxTimeMS: 5000 }).toArray(); // query ko 5s se zyada mat chalne do
}

// ----- Same with pg -----
async function getUserPaidOrderLinesPg(pool, userId) {
  const { rows } = await pool.query(`
    SELECT o.order_id, u.name AS customer, p.name AS product,
      oi.quantity, oi.price, oi.quantity * oi.price AS line_total
    FROM orders o
    JOIN users u      ON u.user_id = o.user_id
    JOIN order_items oi ON oi.order_id = o.order_id
    JOIN products p    ON p.product_id = oi.product_id
    WHERE o.user_id = $1 AND o.status = 'PAID'
    ORDER BY oi.price DESC`, [userId]);
  return rows;
}

// ----- Mongoose -----
const lines = await Order.aggregate([ /* same stages */ ]).option({ maxTimeMS: 5000 });
// Yaad: aggregate() Mongoose schema casting / middleware (find hooks) skip karta hai.

// ----- Prisma – complex aggregation ke liye raw SQL -----
const rows = await prisma.$queryRaw`
  SELECT p.name, SUM(oi.quantity)::int AS units
  FROM order_items oi JOIN products p ON p.product_id = oi.product_id
  GROUP BY p.name ORDER BY units DESC LIMIT ${limit}`; // tagged template → parameterized
```

COMMON MISTAKES

- Saare documents Node.js mein laake JS mein **reduce** se group karna — network + memory waste. DB mein aggregate karo.
- **\$match** ko \$lookup/\$unwind ke baad rakhna jab pehle ho sakta tha.
- User input se pipeline stages banana (**req.body.pipeline**) — injection. Pipeline code mein fixed, sirf values parameter.
- **maxTimeMS** / statement timeout na lagana — ek bhari report query DB ko choke kar sakti hai.
- Mongoose **aggregate** mein string ID se **_id** match — cast nahi hota.
- Har dashboard load pe heavy pipeline — result cache karo ya pre-aggregate (Part 19).

INTERVIEW QUESTION

Q: MongoDB aggregation pipeline ko SQL query se compare karo. Performance ke liye kya dhyan rakhoge?

Answer: Pipeline stages ka sequence hai: \$match≈WHERE, \$lookup≈JOIN, \$unwind≈row expansion, \$group≈GROUP BY, \$project≈SELECT, \$sort≈ORDER BY, \$limit≈LIMIT. SQL declarative hai — planner execution order chunta hai; Mongo pipeline mein likha hua order hi chalta hai. Performance: \$match aur \$sort shuru mein taaki index use ho, \$project se fields kam, \$lookup se pehle \$limit, foreignField pe index, 100 MB stage limit ka dhyan, \$facet se pehle filter, aur explain('executionStats') se verify.

MUST KNOW	GOOD TO KNOW	NOT NECESSARY RIGHT NOW
• Stage → SQL mapping • Complete pipeline likh sako (match, lookup, unwind, group, project, sort, limit) • \$match/\$sort pehle (index) • \$lookup se pehle \$limit	• \$facet (16 MB, no index inside) • CTE (WITH) • COUNT(*) OVER () pagination • maxTimeMS	• \$merge / \$out (materialized results) • \$bucket, \$graphLookup, \$setWindowFields • Pipeline optimizer ke internal rules

Part 8 — Master Memory Table (PDF 2 ka sabse important page)

SQL	MongoDB	Yaad rakhne ki line
WHERE	\$match	Pehle rakho — index
JOIN	\$lookup	Result array
(row expansion)	\$unwind	Array → docs
single match column	\$first / \$arrayElemAt	Array → object
GROUP BY	\$group	_id = key
COUNT / SUM / AVG / MIN / MAX	\$sum:1 / \$sum / \$avg / \$min / \$max	
COUNT(DISTINCT x)	\$addToSet + \$size	
HAVING	\$match (after \$group)	
SELECT a, b AS c	\$project { a: 1, c: "\$b" }	Shape final
SELECT *, x AS y	\$addField / \$set	Baaki fields rakho
ORDER BY	\$sort	1 / -1

SQL	MongoDB	Yaad rakhne ki line
LIMIT / OFFSET	\$limit / \$skip	Pehle sort
SELECT COUNT(*)	\$count	
Multiple queries / COUNT(*) OVER ()	\$facet	16 MB, filter pehle
WITH (CTE)	Pipeline stages	Step-by-step logic

PDF 2 — Final Recap (Parts 5-8)

- Sort dono DBs mein; bina sort koi order guarantee nahi; NULL ordering alag.
- Offset vs cursor pagination — speed, stability, tradeoff; tie-breaker; limit+1; index.
- INNER vs LEFT JOIN; \$lookup = LEFT; empty array; anti-join.
- \$arrayElemAt/\$first vs \$unwind — kab kaunsa; preserveNullAndEmptyArrays.
- LEFT JOIN + WHERE trap; N+1 problem; populate ≠ \$lookup.
- GROUP BY / \$group with 5 aggregates; _id: null; COUNT(*) vs COUNT(col).
- WHERE vs HAVING = \$match before vs after \$group; SQL logical execution order.
- Complete pipeline likhna + equivalent SQL; performance rules; \$facet; CTE.

Practice task (45 min)

- Offset aur cursor dono pagination **GET /orders** pe implement karo — pg aur Mongo repos mein.
- SQL + Mongo dono: (1) har city ke users ka total PAID spend (users + orders join + group by city), (2) jin products ka kabhi koi PAID order nahi hua (anti-join — expected: P102, P104), (3) category-wise revenue (orders → items → products → categories).
- Q3 ka Mongo pipeline pehle \$group phir \$lookup order mein likho, aur socho kyun.